

Backend Series — Module 0 Master Notes

Days 1 → 8 Complete Runtime + Async JavaScript Foundation

WHY THIS MODULE EXISTS

Most people start backend by directly learning:

- Express
- CRUD
- MongoDB
- Auth

But then:

- async bugs destroy them
- middleware flow confusing lagta hai
- promises samajh nahi aate
- race conditions samajh nahi aati
- event loop invisible rehta hai

This module builds:

"How JavaScript ACTUALLY executes code internally"

This is backend foundation.

BIG PICTURE

Backend learning path:

```
JS Basics
↓
Runtime Model
↓
Async Internals
↓
Node.js Runtime
↓
```

Express
↓
DB + Auth + APIs

Current position after Day 8:

Runtime-aware JavaScript learner

DAY 1–3 FOUNDATION SUMMARY

(Brief reconstruction based on progression)

Topics touched:

- JS execution basics
- function execution
- callbacks intuition
- sync vs async intuition
- blocking vs non-blocking intuition
- why async exists

Core idea:

JavaScript executes one thing at a time.

DAY 4 — EXECUTION CONTEXT + CALL STACK

JavaScript Nature

JavaScript engine nature:

Single-threaded
Synchronous

Meaning:

- one thing at a time
 - line by line execution
 - uses call stack
-

CALL STACK

Call stack works using:

LIFO
Last In First Out

Example:

```
function one() {  
  console.log("one");  
}  
  
function two() {  
  one();  
  console.log("two");  
}  
  
function three() {  
  two();  
  console.log("three");  
}  
  
three();
```

Output:

```
one  
two  
three
```

STACK VISUALIZATION

When `three()` runs:

```
| three |  
| global|
```

Then:

```
| two   |  
| three|  
| global|
```

Then:

```
| one   |  
| two   |  
| three |  
| global|
```

Then stack unwinds.

SYNCHRONOUS EXECUTION

Example:

```
console.log("A");  
console.log("B");  
console.log("C");
```

Output:

```
A  
B  
C
```

Reason:

JS waits for current task to complete.

BLOCKING CODE

Example:

```
function heavy() {  
  for (let i = 0; i < 10000000000; i++) {}  
}  
  
console.log("start");  
heavy();  
console.log("end");
```

Output:

```
start  
end
```

BUT delayed.

Reason:

```
Call stack occupied.  
JS cannot move forward.
```

REAL MEANING OF BLOCKING

Definition:

Current task finishes hone tak JS next kaam nahi kar sakti.

Examples:

```
while(true){}
```

```
for(let i=0;i<999999999;i++){}
```

These freeze execution.

FIRST ASYNC INTUITION

Example:

```
console.log("start");

setTimeout(() => {
  console.log("timeout");
}, 0);

console.log("end");
```

Actual output:

```
start
end
timeout
```

IMPORTANT:

```
setTimeout does NOT mean immediate execution.
```

KEY CONCLUSION DAY 4

```
Async callbacks cannot run until synchronous code finishes.
```

DAY 5 — EVENT LOOP + CALLBACK QUEUE + NODE APIs

This was the REAL async foundation.

BIG TRUTH

JavaScript itself is NOT async.

JavaScript engine:

single-threaded
synchronous

Async behavior comes from:

- Browser APIs
 - Node APIs
 - libuv
 - OS systems
-

FULL BASIC ARCHITECTURE

JS Engine
↓
Call Stack
↓
Event Loop
↓
Callback Queue

Side systems:

Web APIs / Node APIs

setTimeout FULL INTERNAL FLOW

Code:

```
console.log("Start");

setTimeout(() => {
  console.log("Timeout");
}, 2000);

console.log("End");
```

Output:

```
Start
End
Timeout
```

INTERNAL EXECUTION FLOW

Step 1

```
console.log("Start")
```

runs on call stack.

Step 2

```
setTimeout(...)
```

Timer registration happens.

IMPORTANT:

```
setTimeout callback executes outside JS engine.
```


Stored in:

```
Node timer system / Web APIs
```

Visualization:

```
NODE TIMER AREA:  
[  
  callback: ()=>console.log("timeout")  
  time: 2000ms  
]
```

Step 3

```
console.log("End")
```

runs immediately.

Step 4

Timer completes.

Callback moves to:

```
CALLBACK QUEUE
```

Step 5

Event loop checks:

```
"Is stack empty?"
```

If yes:

Queue → Stack

Then callback executes.

EVENT LOOP

Definition:

Infinite checker coordinating queues and stack.

Pseudo logic:

```
while(true){  
  if(stack empty){  
    move callback from queue to stack  
  }  
}
```

CALLBACK QUEUE

FIFO structure.

Example:

```
setTimeout(() => console.log("A"), 0);  
setTimeout(() => console.log("B"), 0);
```

Queue order:

A → B

IMPORTANT DOUBTS CLEARED

Is JS blocking?

Clarification:

JS engine executes synchronously.
Blocking/non-blocking describes code behavior.

Is Node.js fully non-blocking?

NO.

Example:

```
while(true){}
```

will freeze Node.

Reason:

JS thread blocked.

Then why Node fast?

Because Node delegates:

- timers
- file system
- network
- database waiting

outside JS thread.

Meanwhile JS remains free.

This is:

Non-blocking I/O

MOST IMPORTANT LINE DAY 5

Async work happens outside JS engine.
Callback execution still happens inside JS engine.

DAY 6 — CALLBACKS + CALLBACK HELL

CALLBACK DEFINITION

Definition:

Function passed into another function to execute later.

Example:

```
function greet(name) {  
  console.log("Hello " + name);  
}  
  
function processUser(cb) {  
  const name = "Harsh";  
  cb(name);  
}  
  
processUser(greet);
```

Output:

Hello Harsh

FUNCTION REFERENCE vs FUNCTION CALL

Reference:

```
greet
```

Call:

```
greet()
```

Huge difference.

CALLBACKS IN ASYNC

Example:

```
setTimeout(() => {  
  console.log("done");  
}, 2000);
```

Callback:

```
() => {  
  console.log("done");  
}
```

WHY CALLBACKS EXIST

Because async result future me aata hai.

Examples:

- file reading
- database queries
- network requests

So:

```
"Kaam complete hone pe ye function chala dena"
```

CALLBACKS ARE NOT AUTOMATICALLY ASYNC

Sync callback

```
function test(cb){  
  cb();  
}
```

Async callback

```
setTimeout(cb, 1000)
```

CALLBACK HELL

Example:

```
getUser(user => {  
  getPosts(user.id, posts => {  
    getComments(posts[0].id, comments => {  
      console.log(comments);  
    });  
  });  
});
```

Structure becomes:

Pyramid of Doom

REAL PROBLEMS OF CALLBACK HELL

NOT just readability.

Main issue:

INVERSION OF CONTROL

INVERSION OF CONTROL

Definition:

You hand over execution responsibility
of your function to someone else.

Example:

```
someLibrary(myCallback)
```

Now library controls:

- when callback runs
- whether it runs
- how many times it runs

REAL RISKS

Callback never called

App stuck.

Callback multiple times

Can cause:

- duplicate payments

- duplicate emails
 - duplicate DB inserts
-

Callback too early

Invalid state.

Callback too late

Unexpected delays.

OLD NODE ERROR-FIRST CALLBACK PATTERN

Example:

```
fs.readFile("a.txt", (err, data) => {  
  
  if(err){  
    console.log(err);  
    return;  
  }  
  
  console.log(data);  
  
});
```

Convention:

```
callback(error,data)
```

KEY CONCLUSION DAY 6

Callback hell's real issue was loss of execution control.

DAY 7 — PROMISES

WHY PROMISES CAME

To solve:

- callback hell
 - inversion of control
 - poor error handling
 - ugly nesting
-

PROMISE DEFINITION

Definition:

Object representing eventual completion or failure of an async operation.

Simple meaning:

"Future me value milegi"

PROMISE STATES

Three states:

Pending

Initial state.

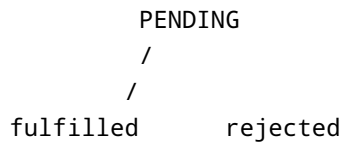
Fulfilled

Success.

Rejected

Failure.

VISUAL



State once settled cannot change.

BASIC PROMISE CREATION

```
const p = new Promise((resolve, reject) => {  
  
  let success = true;  
  
  if(success){  
    resolve("Done");  
  }else{  
    reject("Failed");  
  }  
  
});
```

IMPORTANT:

Promise constructor callback executes immediately.

resolve()

```
resolve("Done")
```

Meaning:

- promise fulfilled
 - value stored
-

reject()

```
reject("Error")
```

Meaning:

- promise rejected
 - error stored
-

CONSUMING PROMISES

Using:

```
.then()  
.catch()
```

SUCCESS EXAMPLE

```
const p = new Promise((resolve)=>{  
  resolve("Success");  
});  
  
p.then((data)=>{  
  console.log(data);  
});
```

Output:

```
Success
```

REJECTION EXAMPLE

```
const p = new Promise((resolve, reject) => {
  reject("Something went wrong");
});

p.catch((err) => {
  console.log(err);
});
```

PROMISE CHAINING

Instead of nested callbacks:

```
getUser()
  .then(user => getPosts(user.id))
  .then(posts => getComments(posts[0].id))
  .then(comments => console.log(comments))
  .catch(err => console.log(err));
```

WHY CHAINING WORKS

Every `.then()` returns NEW promise.

Visualization:

```
Promise
  ↓
  .then()
  ↓
new Promise
```

RETURNING VALUES

```
Promise.resolve(5)
  .then(x => {
    return x * 2;
  })
  .then(x => {
    console.log(x);
  });
```

Output:

10

RETURNING PROMISES

```
fetchUser()
  .then(user => {
    return fetchPosts(user.id);
  })
```

Chain automatically waits.

ERROR PROPAGATION

```
getUser()
  .then(user => getPosts(user.id))
  .catch(err => {
    console.log(err);
  });
```

Any failure:

→ catch

Huge improvement over callbacks.

IMPORTANT CLARITY

Promises DO NOT start async work.

They REPRESENT eventual result.

KEY CONCLUSION DAY 7

Promises restore predictability and structured async flow.

DAY 8 — MICROTASK QUEUE + EVENT LOOP PRIORITY

UPDATED ARCHITECTURE

Actual architecture:

```
Call Stack
↓
Microtask Queue
↓
Callback Queue
↓
Event Loop
```

MICROTASK QUEUE

High-priority queue.

Contains:

- Promise `.then`

- `.catch`
 - `.finally`
 - `queueMicrotask`
-

CALLBACK QUEUE

Contains:

- `setTimeout`
 - `setInterval`
 - DOM events
 - many async callbacks
-

MOST IMPORTANT RULE

Event loop order:

1. Empty call stack
 2. Run ALL microtasks
 3. Run ONE callback queue task
 4. Repeat
-

FAMOUS QUESTION

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

Promise.resolve().then(() => {
  console.log("C");
});

console.log("D");
```

Output:

A
D
C
B

Reason:

Microtask queue priority > callback queue priority

IMPORTANT INTERNAL FLOW

`.then()` callbacks:

→ Microtask Queue

`setTimeout` callbacks:

→ Callback Queue

PROMISE CALLBACKS ARE ALWAYS ASYNC

Even:

```
Promise.resolve().then(()=>{})
```

uses microtask queue.

`.finally()`

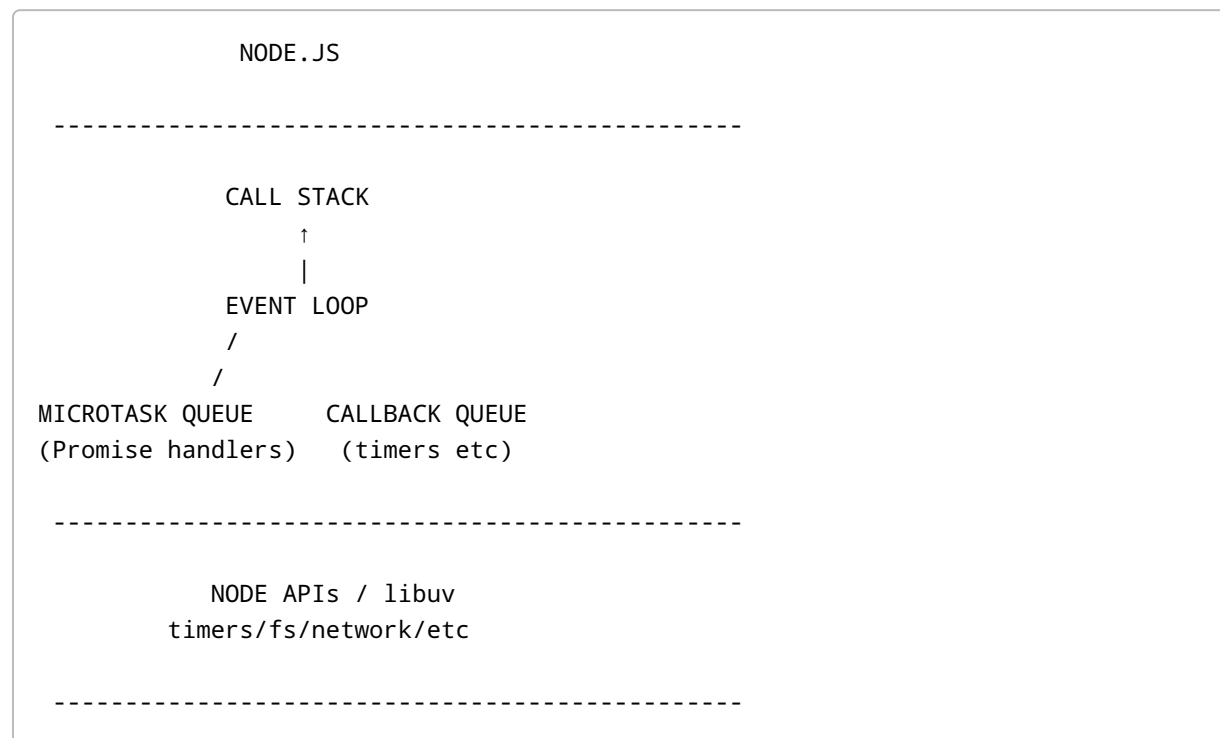
Runs regardless of:

- success
- failure

Example:

```
Promise.resolve("Done")
  .then(data=>{
    console.log(data);
  })
  .finally(()=>{
    console.log("cleanup");
  });
```

FULL FINAL ARCHITECTURE



MOST IMPORTANT INSIGHT DAY 8

Promises are not parallel threads.
They only get higher scheduling priority.

MASTER CLARITY SECTION

JS ENGINE vs NODE.JS

JS Engine

Nature:

single-threaded
synchronous

Job:

execute code on call stack

Node.js

Runtime environment.

Provides:

- APIs
- timers
- file system access
- networking
- event loop
- libuv

BLOCKING vs NON-BLOCKING FINAL CLARITY

Blocking

Call stack occupied continuously.

Examples:

```
while(true){}
```

```
heavy calculations
```

Non-blocking

Work delegated externally.

Examples:

```
setTimeout()
```

```
fs.readFile()
```

```
fetch()
```

setTimeout BIGGEST CLARITY

```
setTimeout(fn,0)
```

DOES NOT mean:

```
"Run immediately"
```

Means:

```
"After minimum wait, place callback into queue"
```

Execution still depends on:

Stack empty or not

WHY THIS FOUNDATION MATTERS FOR BACKEND

These concepts directly connect to:

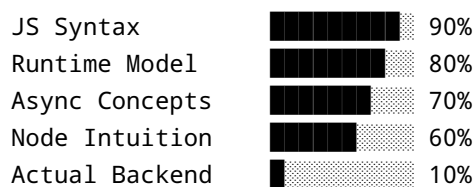
- Express middleware
- database queries
- auth flows
- API handling
- websocket events
- Redis queues
- background jobs
- caching
- performance debugging

Without runtime understanding:

backend becomes memorization.

CURRENT STATUS AFTER DAY 8

Understanding estimate:



MOST IMPORTANT MASTER TAKEAWAYS

Takeaway 1

JavaScript executes one thing at a time.

Takeaway 2

Async work happens outside JS engine.

Takeaway 3

Callbacks execute only when stack becomes free.

Takeaway 4

Promises solve inversion of control.

Takeaway 5

Microtask queue has higher priority than callback queue.

Takeaway 6

Node.js is not magically parallel JS.

Takeaway 7

Backend performance depends heavily on async architecture.

NEXT TOPICS AFTER DAY 8

- async/await

- hidden promises
- await internals
- sequential vs parallel awaits
- Promise combinators
- Node module system
- npm
- Express foundation

Then actual backend begins.